DESIGN REVIEW · V6 · INTERNAL

3GPP UE Root Cause Analysis Skill Suite — Version 6

Per-level iterative Fault Tree Analysis with user-gated top event selection and iteration-by-iteration checkpoints. Built for Cline (VS Code) without MCP.

VERSION	SKILLS
v6.0 (baseline)	12 + 1 workflow + 1 rule

FILES	STATUS
33 (5 shared, 24 skill, 2 control, 2 wrapper)	Built · Pre-test

---

CONTENTS

01 What changed in v6

02 Four-layer architecture

03 Skill catalog

04 End-to-end sequence

05 Iteration model

06 State machine

07 Per-iteration FTA flow

08 Skill I/O reference

09 Python tool I/O reference

10 State file ownership

11 Discussion topics

§01 What changed in v6

v6 introduces **per-level iterative FTA.** v5 ran one fault tree top-to-bottom and emitted one root cause. v6 runs *N iterations* of FTA, each producing an iteration-local root cause; the user decides at each checkpoint whether to dig deeper or accept the current finding as terminal. The pipeline halts at **three to N+1 user gates** per run (one Checkpoint A, N Checkpoint Bs).

V5 MODEL

- One `phase3_hybrid_tree`
- One `phase3_root_cause`
- No user gates (halt-on-error only)
- Top event set automatically
- 10 skills + workflow + rule

V6 MODEL

- `fta_iterations[]` — one per drill level
- `phase3_root_cause_chain` synthesized at terminal
- Checkpoint A (top event) + Checkpoint B per iteration
- Top event chosen from curated `top_event_candidates[]`
- 12 skills (added top-event-confirmation, iteration-controller)

DESIGN DECISIONS LOCKED IN V6.0

D1 · ITERATION BUDGET

5

Default cap; user can override into iteration 6+ via explicit confirmation

D2 · FAST MODE

DROPPED

Every iteration requires user gate — no auto-accept escape valve

D3 · TOP EVENT ALTERNATIVES

≤3 candidates

1 primary + up to 2 defensible alternatives; no padding with hallucinated alts

D4 · USER OVERRIDE

ALLOWED

With explicit "confirm override" prompt; logged with overrode_recommendation flag

D5 · CROSS-ITER CROSS-REF

DEFERRED

Iteration N+1 doesn't cross-ref against iteration N's commanded values — v6.1

D6 · BUILD STRATEGY

FULL REBUILD

All 12 SKILL.md files rewritten from scratch — no patching v5 in place

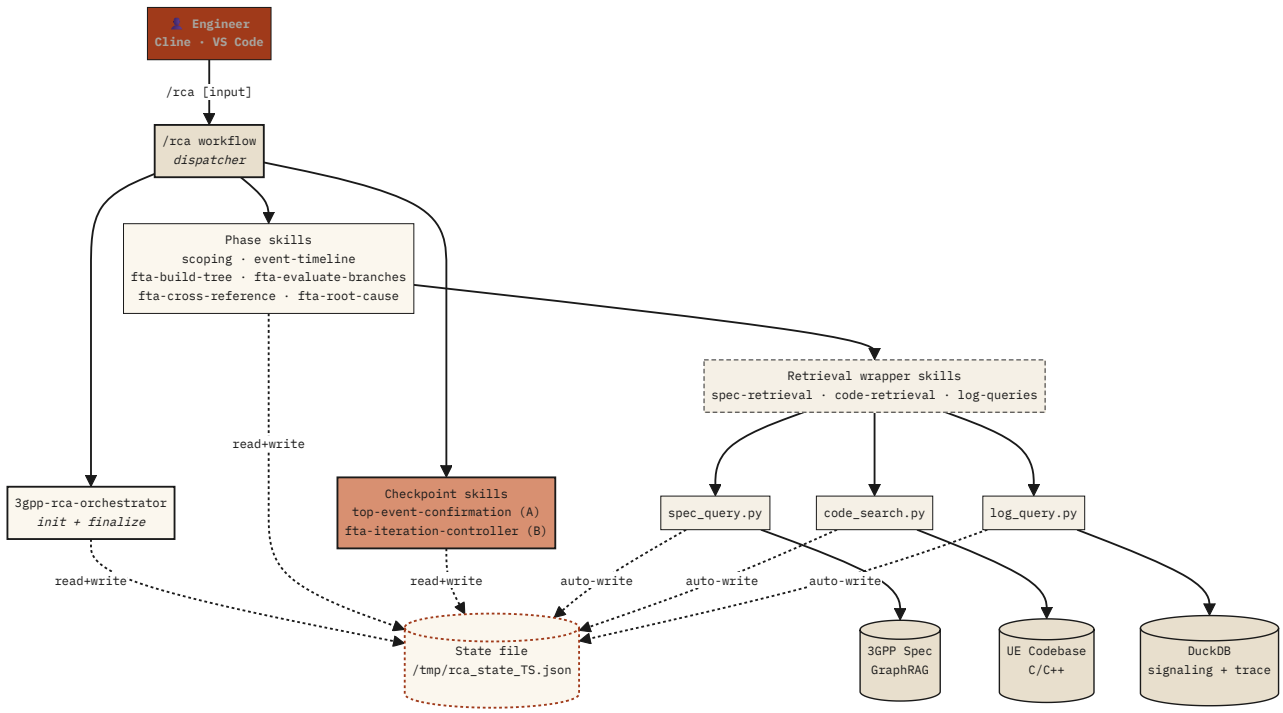
COST HONESTY

A typical 3-iteration run costs roughly **2× the tokens and 3.5× the wall-clock** of an equivalent v5 run. The wall-clock multiplier is dominated by user pause times (one pause per iteration plus Checkpoint A). v6 is the right answer if the failure mode you most want to prevent is the agent *confidently producing wrong root causes*. If your dominant cost is engineer time, v5's autonomous run is faster.

§02 Four-layer architecture

Five layers separated by clear contracts: the engineer interacts only with Cline's `/rca` workflow; the workflow dispatches to skills; skills delegate retrieval to wrapper skills; wrapper skills exec Python scripts; scripts talk to the underlying data stores. The state file is the cross-cutting source of truth — every layer reads and writes to it via slice operations.

FIG. 1 — SYSTEM LAYERS AND DATA FLOW **FIG-01**



WHY A DISPATCHER WORKFLOW?

In v5 the `/rca` workflow ran linearly start-to-finish. In v6 the pipeline must *halt at user gates*. Each `/rca` invocation reads `meta.current_phase` from the state file and runs only the next step — then halts. This makes the pipeline resumable from any checkpoint and lets the user respond to a gate by typing `/rca <option>` in a fresh turn.

§03 Skill catalog

12 skills grouped into four roles. Orchestration runs once at the start and once at the end. Phase skills do the actual analysis work. Checkpoint skills handle user gates. Retrieval skills are thin wrappers around the Python tools.

ORCHESTRATION · 1 SKILL

3gpp-rca-orchestrator	*Phase 0 init + Phase 4 finalize*

CHECKPOINT · 2 SKILLS NEW V6

3gpp-top-event-confirmation	*Checkpoint A — top event*
3gpp-fta-iteration-controller	*Checkpoint B — per iteration*

PHASE · 6 SKILLS

3gpp-scoping	*Phase 1 — IS/IS-NOT scope filter*	V5 VERBATIM
3gpp-event-timeline	*Phase 2 — timeline + top_event_candidates[]*	V6 AMENDED
3gpp-fta-build-tree	*Phase 3.1 — hybrid tree per iteration*	V6 AMENDED
3gpp-fta-evaluate-branches	*Phase 3.2 + 3.3 — Gate A pruning + Gate B verification*	V6 AMENDED
3gpp-fta-cross-reference	*Phase 3.4 — commanded vs actual value*	V6 AMENDED
3gpp-fta-root-cause	*Phase 3.5 — iteration root cause synthesis*	V6 AMENDED

RETRIEVAL WRAPPERS · 3 SKILLS V5 VERBATIM

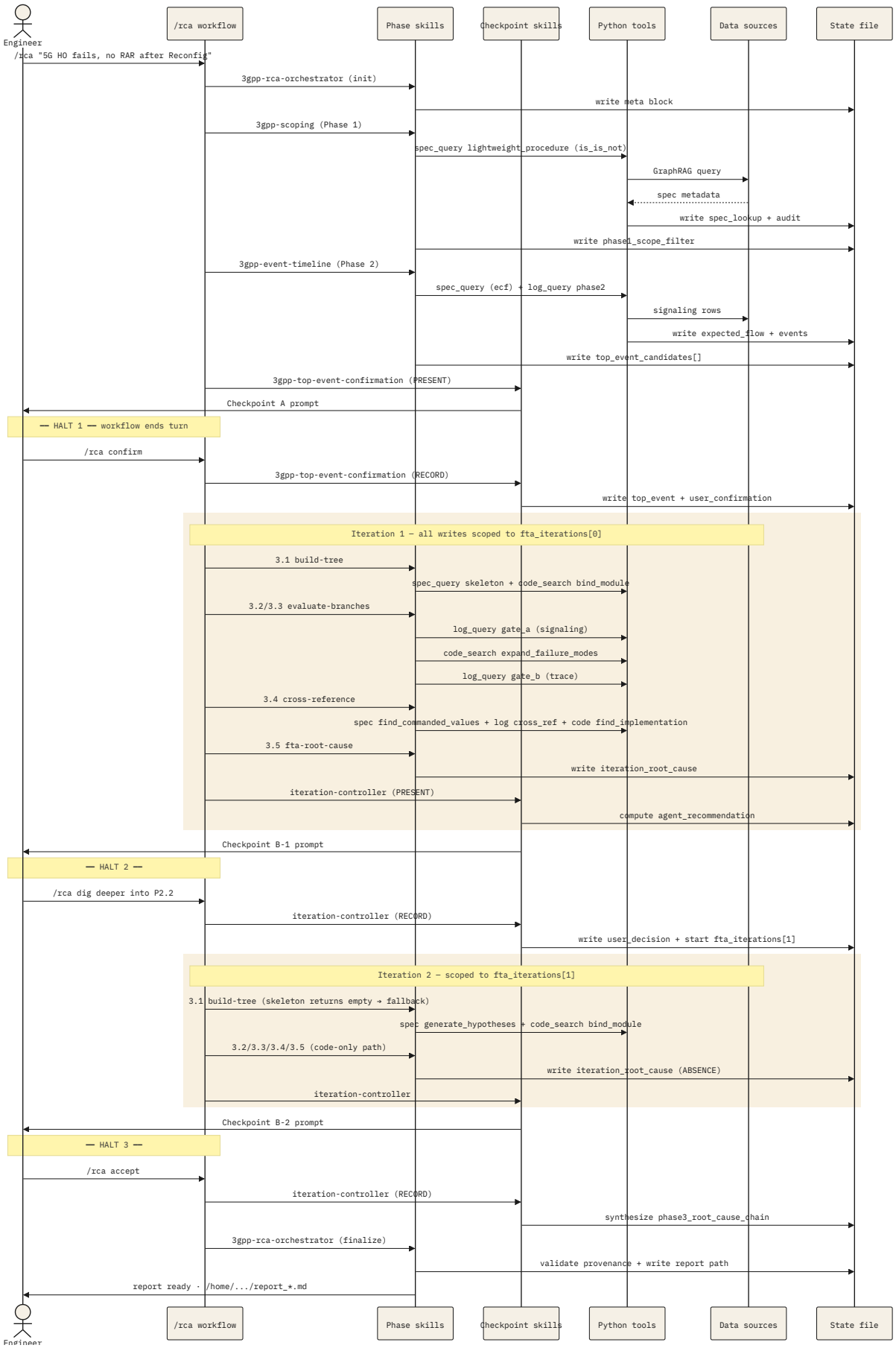
3gpp-spec-retrieval	*Wraps spec_query.py (6 operations)*
3gpp-code-retrieval	*Wraps code_search.py (3 operations)*
3gpp-log-queries	*Wraps log_query.py (5 phase tags)*

Plus one always-on rule (`3gpp-rca-collaboration.md`) and one workflow file (`/rca`), and five shared resources in `_shared/` .

§04 End-to-end sequence

The diagram below shows a realistic two-iteration run from `/rca` trigger to final report. Notice the three HALT points where the workflow ends a turn and waits for the user.

FIG. 2 — END-TO-END SEQUENCE (2-ITERATION EXAMPLE) **FIG-02**


Engineer
/rca workflow
Phase skills
Checkpoint skills
Python tools
Data sources
State file
/rca "5G HO fails, no RAR after Reconfig"
3gpp-rca-orchestrator (init)
write meta block
3gpp-scoping (Phase 1)
spec_query lightweight_procedure (is_is_not)
GraphRAG query
spec metadata
write spec_lookup + audit
write phase1_scope_filter
3gpp-event-timeline (Phase 2)
spec_query (ecf) + log_query phase2
signaling rows
write expected_flow + events
write top_event_candidates[]
3gpp-top-event-confirmation (PRESENT)
Checkpoint A prompt
— HALT 1 — workflow ends turn
/rca confirm
3gpp-top-event-confirmation (RECORD)
write top_event + user_confirmation
Iteration 1 — all writes scoped to fta_iterations[0]
3.1 build-tree
spec_query skeleton + code_search bind_module
3.2/3.3 evaluate-branches
log_query gate_a (signaling)
code_search expand_failure_modes
log_query gate_b (trace)
3.4 cross-reference
spec find_commanded_values + log cross_ref + code find_implementation
3.5 fta-root-cause
write iteration_root_cause
iteration-controller (PRESENT)
compute agent_recommendation
Checkpoint B-1 prompt
— HALT 2 —
/rca dig deeper into P2.2
iteration-controller (RECORD)
write user_decision + start fta_iterations[1]
Iteration 2 — scoped to fta_iterations[1]
3.1 build-tree (skeleton returns empty → fallback)
spec generate_hypotheses + code_search bind_module
3.2/3.3/3.4/3.5 (code-only path)
write iteration_root_cause (ABSENCE)
iteration-controller
Checkpoint B-2 prompt
— HALT 3 —
/rca accept
iteration-controller (RECORD)
synthesize phase3_root_cause_chain
3gpp-rca-orchestrator (finalize)
validate provenance + write report path
report ready · /home/.../report_*.md

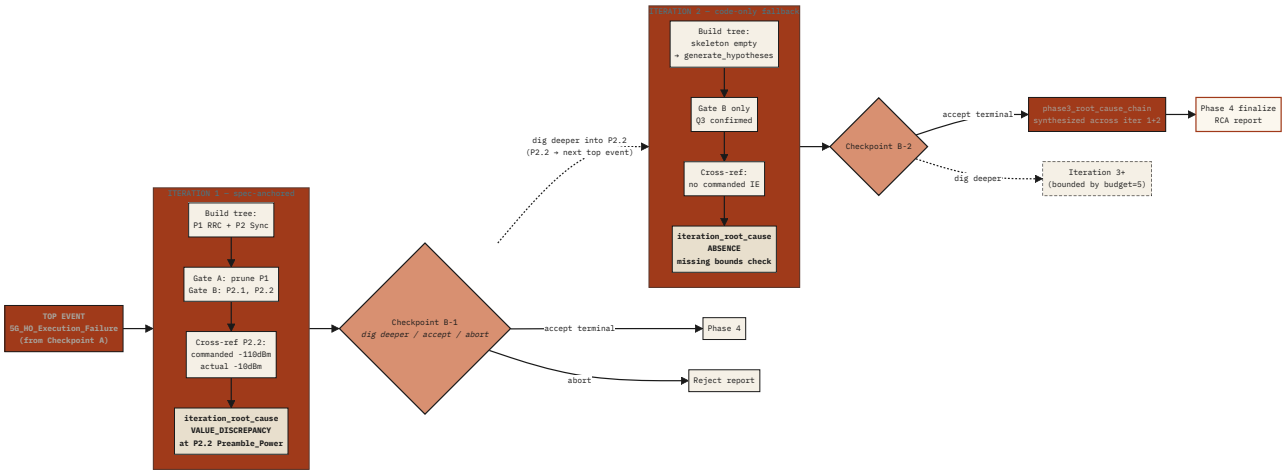
READING THIS DIAGRAM

The three **HALT** annotations are where the agent's turn literally ends. The user must send a new `/rca` message to resume. The state machine in §06 explains how the workflow knows where to resume.

§05 Iteration model

The central concept of v6: **one FTA iteration = one full Phase 3.1→3.5 cycle** over a single input top event, producing one iteration-local root cause. Iterations chain together — iteration N's user-selected base event becomes iteration N+1's input top event.

FIG. 3 — ITERATION CHAINING (CAUSAL CHAIN) FIG-03



WHAT CHANGES AT EACH ITERATION BOUNDARY

ITERATION 1 (TYPICAL)

- `input_top_event.source = phase2_ecf.top_event`
- `spec_anchored: true`
- Skeleton populates branches (spec phases)
- Gate A signaling drives pivot-pruning
- Likely VALUE_DISCREPANCY or MULTI_CAUSE

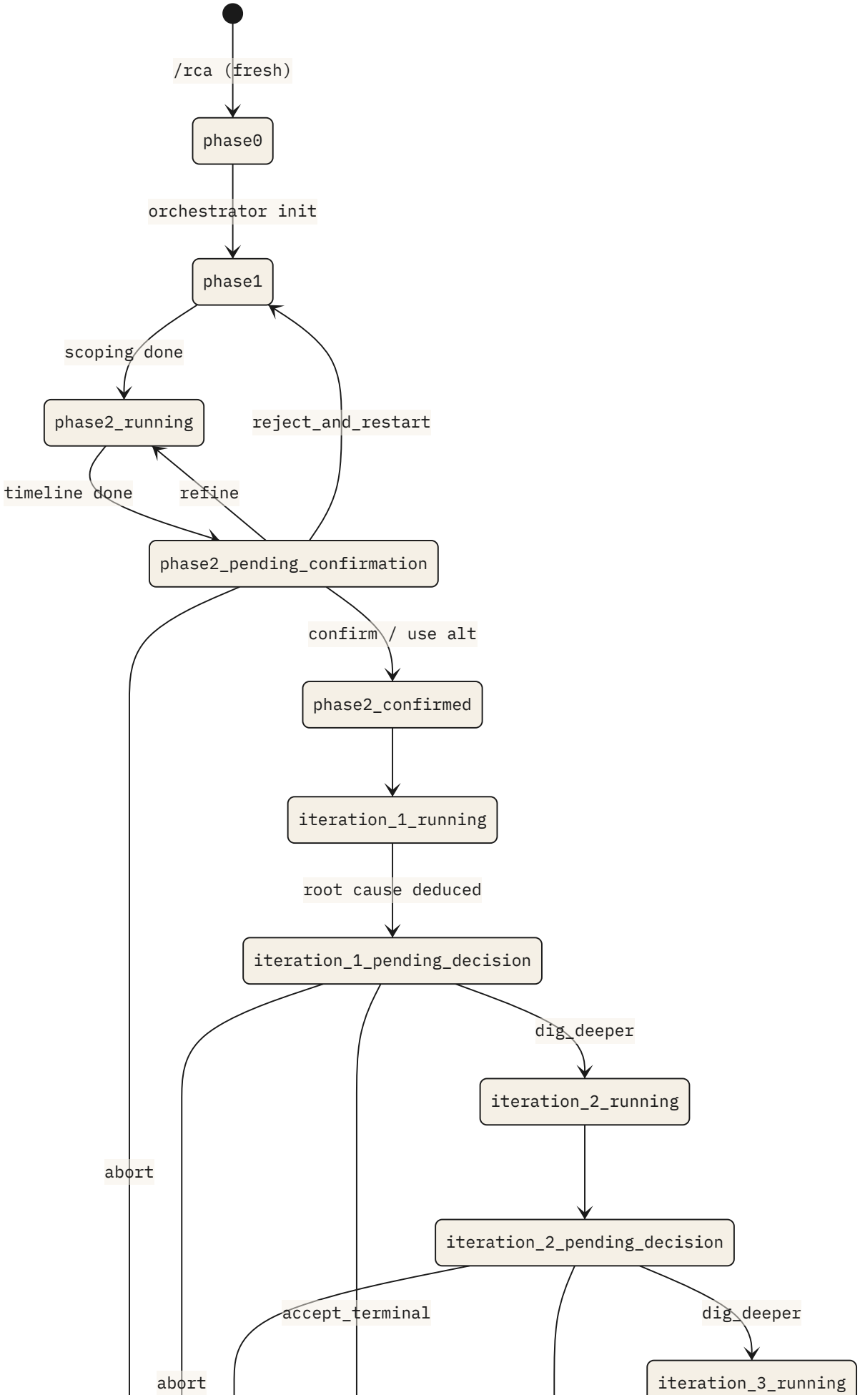
ITERATION ≥ 2 (TYPICAL)

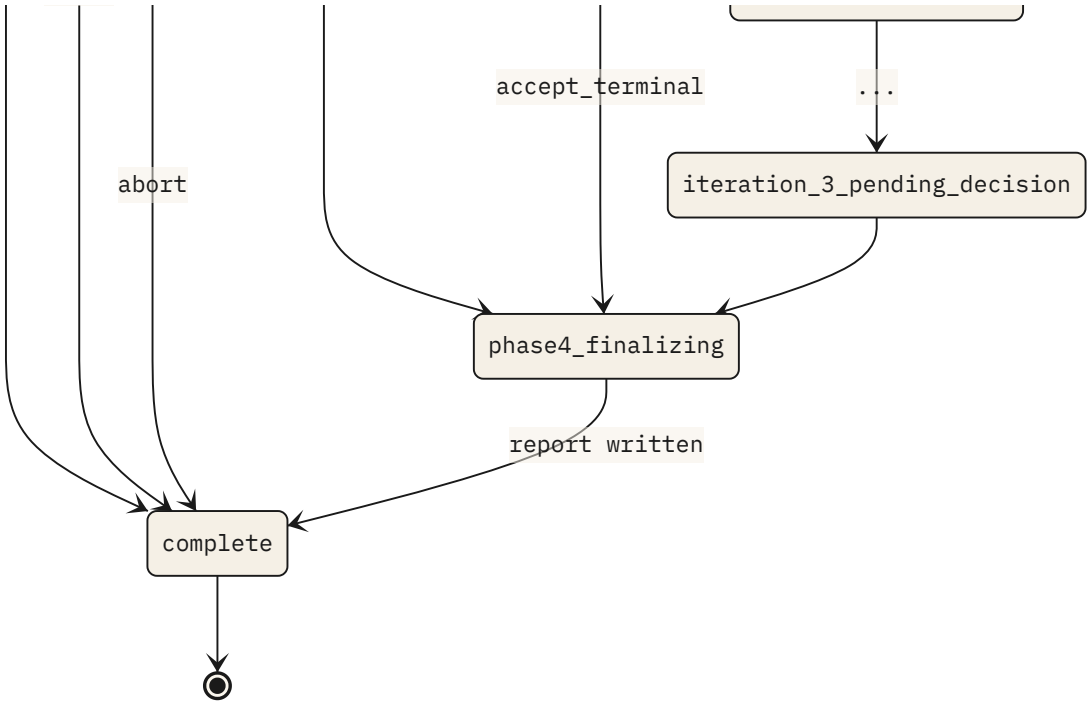
- `input_top_event.source = fta_iterations[N-1].base_events[id]`
- `spec_anchored: false`
- Skeleton often empty → `fallback_used: generate_hypotheses`
- Gate A skipped on code-only branches
- Likely ABSENCE; eventually terminal

§06 State machine

The workflow dispatcher reads `meta.current_phase` from the state file on every `/rca` invocation and runs the next applicable step. Every skill writes the next-phase value before halting or returning.

FIG. 4 — META.CURRENT_PHASE TRANSITIONS **FIG-04**


/rca (fresh)
phase0
orchestrator init
phase1
scoping done
phase2_running
reject_and_restart
timeline done
refine
phase2_pending_confirmation
confirm / use alt
phase2_confirmed
iteration_1_running
root cause deduced
iteration_1_pending_decision
dig_deeper
iteration_2_running
abort
iteration_2_pending_decision
accept_terminal
dig_deeper
abort
iteration_3_running



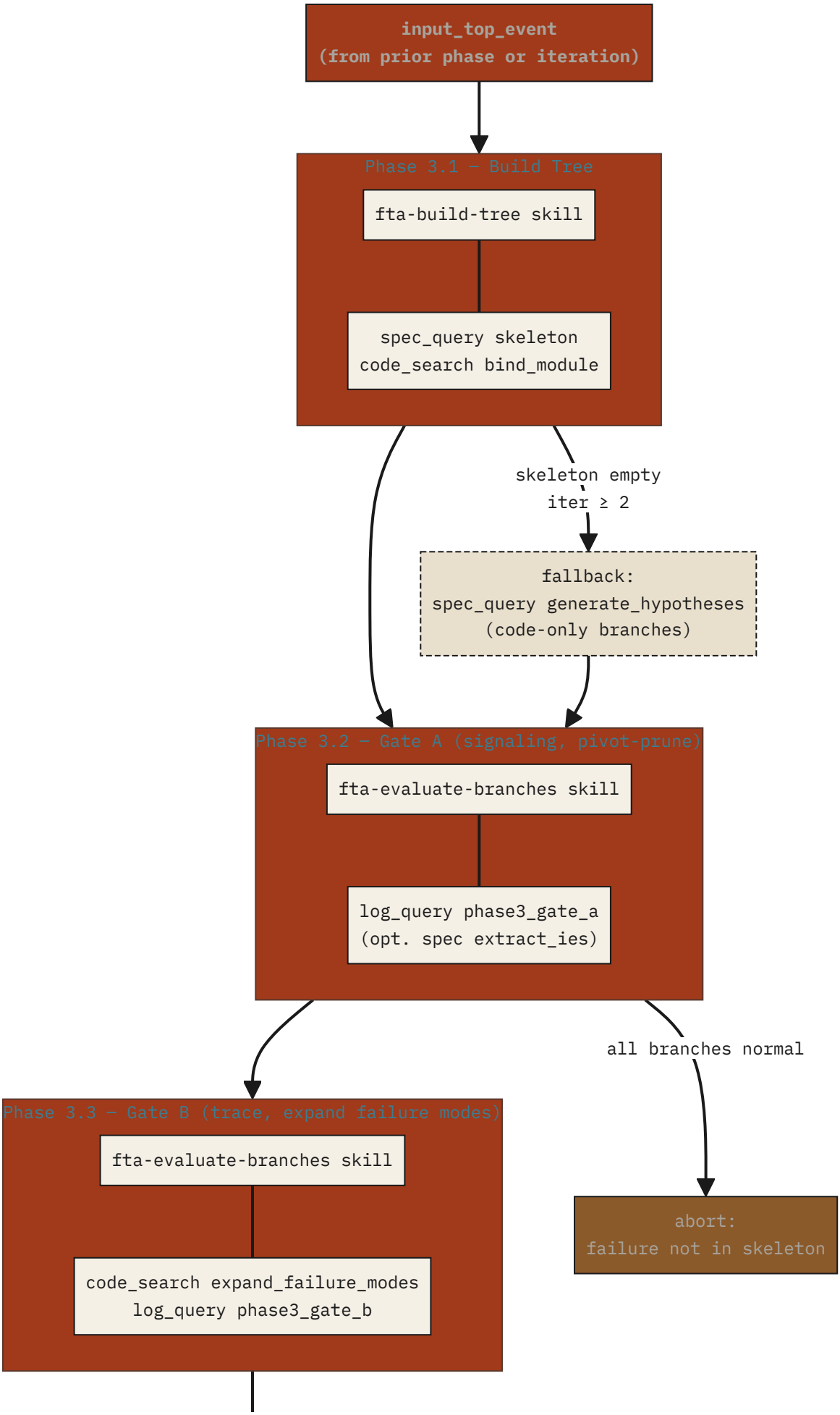
RESUMABILITY

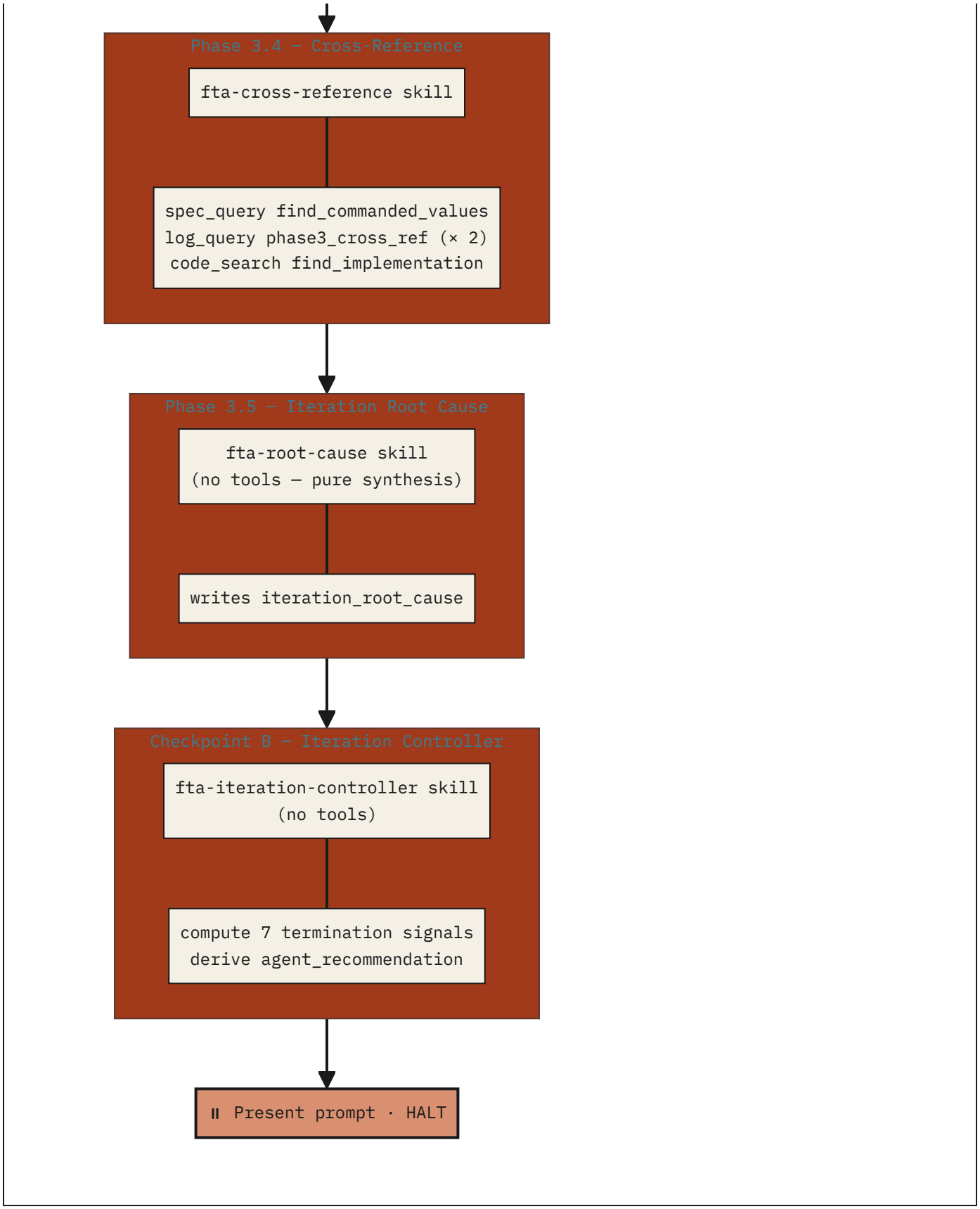
If the Cline task dies mid-iteration, the state file still has `meta.current_phase = "iteration_N_running"` and partial writes in `fta_iterations[N-1]`. The next `/rca` picks up from the last-completed sub-phase. This is why every phase skill is idempotent — it checks what's already in the iteration record before redoing work.

§07 Per-iteration FTA flow

Within a single iteration, five phase sub-steps run in sequence. Below each box: the skill that owns the step and the Python tools it invokes.

FIG. 5 — ONE FTA ITERATION INTERNAL FLOW **FIG-05**


input_top_event
(from prior phase or iteration)
Phase 3.1 — Build Tree
fta-build-tree skill
spec_query skeleton
code_search bind_module
skeleton empty
iter ≥ 2
fallback:
spec_query generate_hypotheses
(code-only branches)
Phase 3.2 — Gate A (signaling, pivot-prune)
fta-evaluate-branches skill
log_query phase3_gate_a
(opt. spec extract_ies)
all branches normal
Phase 3.3 — Gate B (trace, expand failure modes)
fta-evaluate-branches skill
code_search expand_failure_modes
log_query phase3_gate_b
abort:
failure not in skeleton


Phase 3.4 — Cross-Reference
fta-cross-reference skill
spec_query find_commanded_values
log_query phase3_cross_ref (× 2)
code_search find_implementation
Phase 3.5 — Iteration Root Cause
fta-root-cause skill
(no tools — pure synthesis)
writes iteration_root_cause
Checkpoint B — Iteration Controller
fta-iteration-controller skill
(no tools)
compute 7 termination signals
derive agent_recommendation
⏸ Present prompt · HALT

ANTI-LAZINESS INVARIANTS (PRESERVED FROM V3+)

- **All branches evaluated** – Gate A processes every top-level branch even if one already shows failure
- **All failure modes evaluated** – Gate B processes every expanded failure mode even if one is confirmed
- **Table isolation enforced at the script layer** – log_query.py refuses Phase 1/2 queries against UE_Trace_log
- **Keyword provenance auto-appended** – every tool invocation logs which keywords it produced and which iteration they belong to

§08 Skill I/O reference

What each skill reads from the state file, what it writes back, and which Python tools it invokes. This is the contract surface — any change to a skill must preserve these I/O shapes or all downstream consumers break.

SKILL	PHASE	INPUT (READS FROM STATE FILE)
3gpp-rca-orchestrator INIT MODE	0	engineer_input (from CLI) duckdb_path, tool_dir
3gpp-scoping	1	meta.engineer_input
3gpp-event-timeline V6	2	phase1_scope_filter
3gpp-top-event-confirmation CHKPT A	A	phase2_ecf.top_event_candidates phase2_ecf.observable_symptoms.log_coverage user_action (RECORD mode only)
3gpp-fta-build-tree V6	3.1	iteration_id (param) phase1_scope_filter fta_iterations[iter-1].input_top_event

SKILL	PHASE	INPUT (READS FROM STATE FILE)
3gpp-fta-evaluate-branches V6	3.2/3.3	iteration_id (param) fta_iterations[iter-1].hybrid_tree phase2_ecf.observable_symptoms.missing_ever
3gpp-fta-cross-reference V6	3.4	iteration_id (param) fta_iterations[iter-1].base_events[] fta_iterations[iter-1].hybrid_tree (parent branches)
3gpp-fta-root-cause V6	3.5	iteration_id (param) full fta_iterations[iter-1] slice (tree, base_events, cross_ref findings, etc
3gpp-fta-iteration-controller CHKPT B	B	iteration_id (param) fta_iterations[iter-1] (full) all prior fta_iterations[*] (for cross-iter signals) meta.iteration_budget user_action (RECORD mode)
3gpp-rca-orchestrator FINALIZE MODE	4	Full state file (only place a full read occurs)

SKILL	PHASE	INPUT (READS FROM STATE FILE)
		——— Retrieval wrapper skills (thin pass-t
3gpp-spec-retrieval WRAPPER	multi	operation + op-specific args (from caller)
3gpp-code-retrieval WRAPPER	multi	operation + op-specific args (from caller)
3gpp-log-queries WRAPPER	multi	phase_tag + table + keywords + filters (fro caller)

STATE FILE SLICE-READ DISCIPLINE

Every skill reads *only* the slice it needs. Loading the full state file is reserved for orchestrator finalize. This keeps token usage proportional to the iteration depth, not to total run history. A 5-iteration run with a Phase 3 retry should still fit in the context window because no single skill needs to see more than its own iteration plus the immediate predecessor.

§09 Python tool I/O reference

Each Python tool exposes multiple *operations* selected by a CLI flag. Every invocation receives `--state-file` and writes structured output to the relevant section directly; stdout returns a compressed JSON summary for the skill to inspect. **Tool scripts also auto-append to `keyword_provenance_audit`** — this is how anti-hallucination invariants are enforced mechanically rather than by skill discipline.

SPEC_QUERY.PY — 3GPP SPEC GRAPHRAG (6 OPERATIONS)

--OPERATION	REQUIRED ARGS	STDOUT SHAPE (COMPRESSED)	STATE FILE DESTINATION
skeleton	--procedure --rat --top-event	phases[] (id, name, spec_ref, mandatory_messages, layer) gate_at_top, spec_refs[]	fta_iterations[N-1].hyb .spec_skeleton_source
lightweight_procedure	--procedure --rat --need is_is_not | ecf	is_is_not: primary_layers, key_timers, initiating_message, spec_refs ecf: expected_flow[order, message, direction, layer]	is_is_not → phase1_scope_filter.spe ecf → phase2_ecf.observable_s .expected_flow_source
extract_ies	--message --procedure --spec-ref --hypothesis-id	message_definitions[] (name, mandatory_ies, optional_ies, direction)	phase3_evaluations[id] .spec_ie_extraction

--OPERATION	REQUIRED ARGS	STDOUT SHAPE (COMPRESSED)	STATE FILE DESTINATION
find_commanded_values	--base-event-name --base-event-layer --upstream-messages	commanded_value_ies[] (ie_name, message, meaning, spec_ref, ue_action, range_unit)	fta_iterations[N-1].cross_reference_findings[id].commande
generate_hypotheses *fallback*	--event --procedure --rat	hypotheses[] (id, cause, spec_ref, gate, ie_names, message_names, layer)	fta_iterations[N-1].hyb (when skeleton empty)
expand_sub_causes *fallback*	--parent-cause --parent-spec-ref --procedure	sub_causes_exist (bool) sub_causes[] (cause, spec_ref, gate, ie_names, message_names)	fta_iterations[N-1].hyb .branches[id].children

CODE_SEARCH.PY — UE CODEBASE SEMANTIC SEARCH (3 OPERATIONS)

--OPERATION	REQUIRED ARGS	STDOUT SHAPE (COMPRESSED)	STATE FILE DESTINATION
bind_module	--phase-name --phase-ref --scope-layer	modules[] (file, primary_function, relevance, evidence)	fta_iterations[N-1].hybri .branches[id].modules

--OPERATION	REQUIRED ARGS	STDOUT SHAPE (COMPRESSED)	STATE FILE DESTINATION
expand_failure_modes	--module --phase-context	failure_modes[] (id_suffix, name, detection_code, log_keywords*, log_tags*, function_signature, layer) * literal macro strings, NEVER inferred	fta_iterations[N-1].hybri .branches[id].children[]
find_implementation	--hypothesis-cause --spec-ie-names --scope-procedure --scope-layer	found (bool), confidence log_keywords*, log_tags* function_names, implementation_files evidence_summary	fta_iterations[N-1].cross_reference_findings[id].actual_valu

LOG MACRO LITERAL EXTRACTION (CRITICAL)

All `log_keywords` returned by `code_search.py` are **literal strings** extracted from these exact macros — never inferred from naming conventions:

```
MSG_HIGH(...) · NAS_MSG_HIGH(...) · LOG_MSG(code, ...) · QCRIL_LOG_DEBUG(...) ·
RRC_LOG_MSG(level, ...) ·
SYS_ERR(...) · DS_MSG_HIGH_1(...) · printf("[TAG] ...") · fprintf(stderr, ...)
· syslog(level, ...)
```

LOG_QUERY.PY — DUCKDB QUERY (5 PHASE TAGS)

Table isolation is enforced at script entry. Mismatched `--phase-tag` + `--table` combinations exit with code 3 and a policy violation JSON. The script *refuses* the query rather than letting downstream skills bypass the rule.

--PHASE-TAG	ALLOWED --TABLE	CALLER SKILL	--RET VALUE
phase1	UE_3gpp_signaling_log **ONLY**	3gpp-scoping	no
phase2	UE_3gpp_signaling_log **ONLY**	3gpp-event-timeline	no
phase3_gate_a	UE_3gpp_signaling_log **ONLY**	3gpp-fta-evaluate-branches	no
phase3_gate_b	UE_Trace_log **ONLY**	3gpp-fta-evaluate-branches	no
phase3_cross_ref	EITHER (caller specifies)	3gpp-fta-cross-reference	**yes** (

EXIT CODE CONVENTIONS (UNIFORM ACROSS ALL THREE PYTHON TOOLS)

0	Success — stdout valid JSON, state file updated
1	Invalid args (bug in calling skill — halt with error)
2	Tool unavailable (backend down — halt pipeline)
3	Policy violation (e.g. table isolation — never retry)
4	Empty result (valid; caller decides fallback)

§10 State file ownership

Single state file at `/tmp/rca_state_<ts>.json`. Path is stored at `.rca/current_state_path.txt`. Every section has exactly one write owner (multiple readers); this matrix is the cross-reference for any debugging session.

STATE FILE SECTION	WRITTEN BY
`meta.*`	3gpp-rca-orchestrator (init)
`meta.current_phase`	Every skill on entry/exit
`phase1_scope_filter`	3gpp-scoping
`phase2_ecf.top_event`	3gpp-top-event-confirmation
`phase2_ecf.observable_symptoms`	3gpp-event-timeline
`phase2_ecf.top_event_candidates[]`	3gpp-event-timeline V6
`phase2_ecf.user_confirmation`	3gpp-top-event-confirmation V6
`user_decisions[]`	3gpp-top-event-confirmation, 3gpp-fta-iteration-controller V6

STATE FILE SECTION	WRITTEN BY
`fta_iterations[i].input_top_event`	3gpp-rca-orchestrator (iter 1) *or* 3gpp-fta-iteration-controller (iter ≥2)
`fta_iterations[i].hybrid_tree`	3gpp-fta-build-tree
`fta_iterations[i].pruned_branches, base_events, rejected, open_items`	3gpp-fta-evaluate-branches
`fta_iterations[i].cross_reference_findings[]`	3gpp-fta-cross-reference
`fta_iterations[i].iteration_root_cause`	3gpp-fta-root-cause
`fta_iterations[i].agent_recommendation`	3gpp-fta-iteration-controller V6
`fta_iterations[i].user_decision`	3gpp-fta-iteration-controller V6
`phase3_root_cause_chain`	3gpp-fta-iteration-controller (on accept_terminal) V6

STATE FILE SECTION	WRITTEN BY
phase4_rca_report	3gpp-rca-orchestrator (finalize)
keyword_provenance_audit[]	All three Python tools (auto-append)

§11 Discussion topics for the review

These are open risks and design choices worth re-examining with the team. None of them are blockers for v6.0 — but each is a place where v6 could surprise us in production, and the team's experience may shift our position.

Q1 — RUBBER-STAMPING RISK

WILL ENGINEERS ACTUALLY ENGAGE WITH CHECKPOINTS, OR JUST TYPE "CONFIRM" AND MOVE ON?

v6's value proposition rests on engineers reviewing the agent's evidence at each checkpoint. If review becomes a reflex acceptance, we've added user-pause cost without buying snowball-error prevention. The checkpoint presentation format (`_shared/checkpoint-presentation-formats.md`) shows full evidence and rationale to make engaged review easier than blind acceptance — but discipline still matters. **Question for the team:** do we need additional friction (e.g. requiring a rationale string when accepting) for high-stakes runs?

Q2 — ITERATION BUDGET AND RUNAWAY PREVENTION

IS 5 THE RIGHT DEFAULT? IS THE BUDGET-NEAR-EXHAUSTION HEURISTIC STRONG ENOUGH?

Default 5 should cover typical depth-2 to depth-4 chains comfortably. The controller's recommendation logic forces `accept_terminal` when budget-1 is reached, but users can still override. **Question:** should we treat budget exhaustion as a hard wall (no override allowed) for non-admin engineers, or trust the override-confirmation flow?

Q3 — CODE-IMPLEMENTATION FLOOR DETECTION

THE CONTROLLER TERMINATES WHEN ≥2 SIGNALS FIRE (SAME FILE ACROSS ITERS, SINGLE-BRANCH TREE, NO COMMANDED IES, ETC). IS THE HEURISTIC RIGHT?

Termination signals are mechanical, listed in `references/recommendation-logic.md`. In testing we may find that some real failures naturally fire 2+ signals at iteration 2 (e.g. a clear PHY bug isolated to one module). That's fine — accept_terminal is the right recommendation, user can still dig if needed. **Question:** should we keep this set of 7 signals as v6.0 baseline, or wait for real-run data before locking them in?

Q4 — CROSS-ITERATION CROSS-REFERENCE (D5 DEFERRED)

V6 BASELINE DOES NOT CROSS-REFERENCE ITERATION N'S FINDINGS AGAINST ITERATION N-1'S COMMANDED VALUES. SHOULD V6.1 ADD THIS?

Scenario: iter 1 finds VALUE_DISCREPANCY at P2.2 (commanded -110 dBm, actual -10 dBm). User digs into P2.2; iter 2 finds an ABSENCE in `dsp_calc_preamble_tx_power`. Today, iter 2 does *not* automatically re-verify that the iter 1 commanded value was correctly received by the DSP layer. **Question:** is this gap real (worth v6.1 work) or theoretical (rarely matters in practice)?

Q5 — HARD TERMINATION AT VERIFIED ROOT CAUSE

V3+ INVARIANT: PIPELINE NEVER PRODUCES FIX RECOMMENDATIONS. STILL RIGHT FOR V6?

The boundary remains: pipeline produces root cause + causal chain + evidence + audit; downstream engineering processes own fix design. Phase 4 orchestrator validates that no string in the state file contains "fix:", "recommendation:", "patch:", etc. **Question:** any pressure from product/customers to soften this and have the agent propose code changes? If yes, that's a different product, not a v6.1.

Q6 — STATE FILE LIFECYCLE

STATE FILES AT `/TMP/` ACCUMULATE. CLEANUP POLICY?

v6 doesn't auto-delete state files after completion — they're preserved for audit. `.rca/current_state_path.txt` points to the most recent. Long-running engineers

may end up with dozens of historical state files. **Question:** add a `/rca cleanup` sub-command? Or rely on OS tmp cleanup? Or move state file out of `/tmp/` entirely?

Q7 — FAILURE RECOVERY FROM PARTIAL ITERATION

IF CLINE CRASHES MID-ITERATION (BETWEEN GATE A AND GATE B), DOES THE NEXT /RCA PICK UP CORRECTLY?

State machine is `iteration_N_running` during the entire iteration. Workflow dispatches by re-running the iteration from build-tree; phase skills are designed to be idempotent (they check if their output is already populated before redoing work). **Question for testing:** we should verify this empirically — what's the smoke test for crash-resume?

Q8 — MOBILE / SHARED REVIEW

RCA REPORTS INCLUDE CAUSAL CHAINS ACROSS ITERATIONS. ARE THEY READABLE FOR SOMEONE NOT IN THE RUN?

Report template (`_shared/rca-report-template.md`) renders Section 4 as the chain plus per-iteration detail in Section 3. Length: 5–7 pages for a 3-iteration run. **Question:** should we add a 1-page TL;DR header (final root cause + chain only) for engineers who just want the punch-line?

---

v6 Design Review · 3GPP UE RCA Skill Suite
Single-file HTML · Mermaid via CDN

Files: `/mnt/skills/user/v6/` · 33 files
Drive: `3gpp-skills-v6` root folder